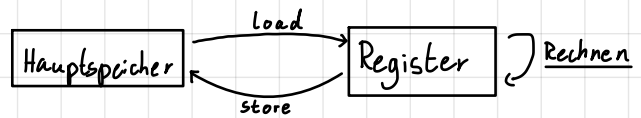


ERA Tutorium 3

Leopold Cario	<u>Termin:</u>	<u>Raum:</u>	<u>Zulip:</u>
	· Mittwoch 10:00	03.13.010	ERA Tutorium - Mi - 1000 - 1
	· Donnerstag 12:00	00.08.059	ERA Tutorium - Do - 1200 - 1

Load and Store Architecture:



Load/Store Befehle

- Load und Store in 8 (b), 16 (h), 32 (w) Bit Varianten
- Bei 8 und 16 Bit-Load Varianten für Sign- und Zero-Extension des geladenen Datums
- effektive Adresse = $rs1 + imm$
- nur Register als Quelle für Adresse (keine Immediates!)

op	funct3	funct7	Type	Instruction	Description	Operation
0000011 (3)	000	-	I	lb rd, imm(rs1)	load byte	$rd = \text{SignExt}([Address]_{7:0})$
0000011 (3)	001	-	I	lh rd, imm(rs1)	load half	$rd = \text{SignExt}([Address]_{15:0})$
0000011 (3)	010	-	I	lw rd, imm(rs1)	load word	$rd = [Address]_{31:0}$
0000011 (3)	100	-	I	lbu rd, imm(rs1)	load byte unsigned	$rd = \text{ZeroExt}([Address]_{7:0})$
0000011 (3)	101	-	I	lhu rd, imm(rs1)	load half unsigned	$rd = \text{ZeroExt}([Address]_{15:0})$
0100011 (35)	000	-	S	sb rs2, imm(rs1)	store byte	$[Address]_{7:0} = rs2_{7:0}$
0100011 (35)	001	-	S	sh rs2, imm(rs1)	store half	$[Address]_{15:0} = rs2_{15:0}$
0100011 (35)	010	-	S	sw rs2, imm(rs1)	store word	$[Address]_{31:0} = rs2$

Byte-adressierbarer Speicher:

n	n+1	n+2	n+3	...

$a0 = 0x \text{12 34 56 78}$ $a1 = n$

$sw \ a0, \ 0(a1)$

n	n+1	n+2	n+3	...
78	56	34	12	

$\Rightarrow$ Little Endian

$lbu \ a2, \ 0(a1) : \ a2 = 0x78$

$lhu \ a2, \ 0(a1) : \ a2 = 0x5678$

$lhu \ a2, \ 2(a1) : \ a2 = 0x1234$

$lw \ a2, \ 0(a1) : \ a2 = 0x12345678$

Lehrstuhl für
Rechnerarchitektur & Parallele Systeme
Prof. Dr. Martin Schulz
Dominic Prinz
Jakob Schäffeler

Lehrstuhl für
Design Automation
Prof. Dr.-Ing. Robert Wille
Stefan Engels
Benjamin Hien

Einführung in die Rechnerarchitektur

Wintersemester 2025/2026

Übungsblatt 3: RISC-V Teil 2 – Sprünge und Pointer 03.11.2025 – 07.11.2025

1 Arrays und deren Adressierung

Felder (engl. arrays) sind ein wichtiger Bestandteil vieler Programme. Dabei werden Daten in einem linearen, zusammenhängenden Speicherbereich organisiert. Das heißt es wird ein Element nach dem anderen im Speicher abgelegt.

- a) Überlegen Sie sich eine Formel mit der Sie die Adresse des n -ten Elementes in Abhängigkeit der Startadresse des Arrays für 64-Bit große Elemente ausrechnen können.

S	S+8	S+16	S+24
0	1	2	3

$\text{double element} = \text{array}[n]$

$\text{Start} + 8 \cdot n$

- b) Wie ändert sich diese Formel, wenn sich die Größe der Elemente ändert?

$\text{Start} + \text{size} \cdot n$

- c) Im Speicher liege ein Feld von 64-Bit Zahlen. Die Startadresse des Feldes (=Pointer) stehe im Register `a0`.

Schreiben Sie ein Unterprogramm, welches das `a1`-te Element mit dem nachfolgenden tauscht. Sie dürfen annehmen, dass das Feld ausreichend lang ist. Das Startelement hat (wie in der Informatik üblich) den Index 0.

swap_64:

; a0: Startadresse

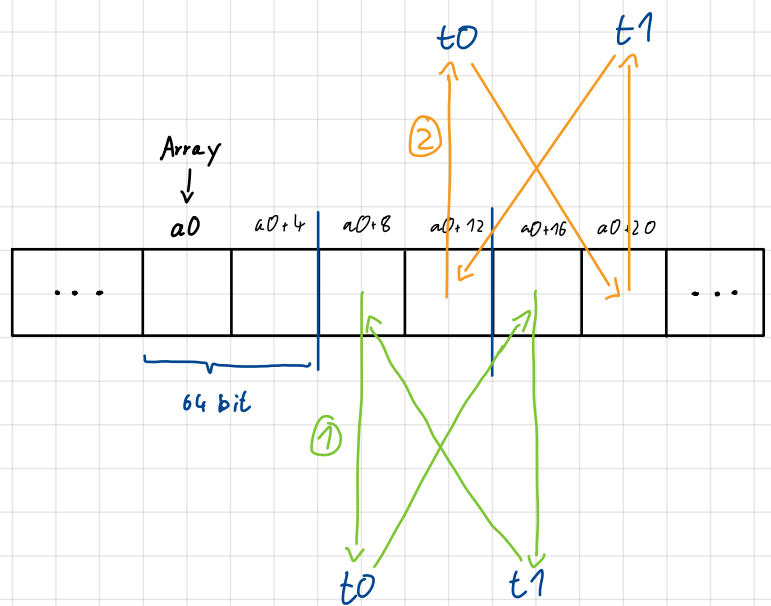
; a1: zu tauschendes Element

slli a1, a1, 3 # a1 = 8·n
add a1, a0, a1 # a1 = 5 + 8·n

lw t0, 0(a1)
lw t1, 8(a1) ①
sw t0, 8(a1)
sw t1, 0(a1)

lw t0, 4(a1)
lw t1, 12(a1) ②
sw t0, 12(a1)
sw t1, 4(a1)

jalr zero, 0(ra)



Sprünge:

1100011 (99)	000	-	B	beq rs1, rs2, label	branch if =	if (rs1 == rs2) PC = BTA
1100011 (99)	001	-	B	bne rs1, rs2, label	branch if ≠	if (rs1 ≠ rs2) PC = BTA
1100011 (99)	100	-	B	blt rs1, rs2, label	branch if <	if (rs1 < rs2) PC = BTA
1100011 (99)	101	-	B	bge rs1, rs2, label	branch if ≥	if (rs1 ≥ rs2) PC = BTA
1100011 (99)	110	-	B	bltu rs1, rs2, label	branch if < unsigned	if (rs1 < rs2) PC = BTA
1100011 (99)	111	-	B	bgeu rs1, rs2, label	branch if ≥ unsigned	if (rs1 ≥ rs2) PC = BTA
1100111 (103)	000	-	I	jalr rd, rs1, imm	jump and link register	PC = rs1 + SignExt(imm), rd = PC + 4
1101111 (111)	-	-	J	jal rd, label	jump and link	PC = JTA, rd = PC + 4

j label ≡ jal zero, label

2 for-wissen

for-Schleifen sind ein weitverbreitetes Konzept aus Hochsprachen wie C oder Java. Eine typische for-Schleife in C sieht wie folgt aus:

```
for (int i = 0; i < 10; i++) {  
    // Schleifenkörper  
}
```

Diese Schleife lässt sich in vier Teile zerlegen:

- **Initialisierung:** `int i = 0;`
Dieser Teil wird einmalig zu Beginn ausgeführt und initialisiert die Schleifenvariable.
- **Bedingung:** `i < 10;` $\hat{=}$ `if i >= 10 : break`
Diese Bedingung wird vor jeder Iteration geprüft. Ist sie wahr, wird der Schleifenkörper ausgeführt, ansonsten wird die Schleife beendet.
- **Inkrement:** `i++`
Dieser Teil wird am Ende jeder Iteration ausgeführt und erhöht (oder erniedrigt) typischerweise die Schleifenvariable.
- **Schleifenkörper:** `// Schleifenkörper`
Der eigentliche Code, der wiederholt ausgeführt wird.

Übersetzen Sie die allgemeine Form einer for-Schleife `for (int i = 0; i < N; i++)` in RISC-V Assembly. Nehmen Sie dabei an, dass N in a0 und i in t0 gespeichert ist.

li t0, 0

loop-start:

bge t0, a0, finished

// Body

addi t0, t0, 1

j loop-start

finished:

:

3 Zeichenketten/Strings

Auch um Strings darzustellen können Arrays verwendet werden. Um einen String abzuspeichern wird jedes Zeichen einzeln als 7-Bit Zahl kodiert (=ASCII-Code) und diese dann fortlaufend als Bytes (8-Bit) abgespeichert. Dabei ist das höchstwertigste Bit eines Bytes immer Null.

ASCII (1977/1986)
ISO/IEC 646:1991

	x0	x1	x2	x3	x4	x5	x6	x7	x8	x9	xA	xB	xC	xD	xE	xF
0x	0 0x0 NUL	1 0x1 SOH	2 0x2 STX	3 0x3 ETX	4 0x4 EOT	5 0x5 ENQ	6 0x6 ACK	7 0x7 BEL	8 0x8 BS	9 0x9 HT	10 0xA LF	11 0xB VT	12 0xC FF	13 0xD CR	14 0xE SO	15 0xF SI
1x	16 0x10 DLE	17 0x11 DC1	18 0x12 DC2	19 0x13 DC3	20 0x14 DC4	21 0x15 NAK	22 0x16 SYN	23 0x17 ETB	24 0x18 CAN	25 0x19 EM	26 0x1A SUB	27 0x1B ESC	28 0x1C FS	29 0x1D GS	30 0x1E RS	31 0x1F US
2x	32 0x20 SPACE	33 0x21 !	34 0x22 "	35 0x23 #	36 0x24 \$	37 0x25 %	38 0x26 &	39 0x27 '	40 0x28 (	41 0x29)	42 0x2A *	43 0x2B +	44 0x2C ,	45 0x2D -	46 0x2E .	47 0x2F /
3x	48 0x30 0	49 0x31 1	50 0x32 2	51 0x33 3	52 0x34 4	53 0x35 5	54 0x36 6	55 0x37 7	56 0x38 8	57 0x39 9	58 0x3A :	59 0x3B ;	60 0x3C <	61 0x3D =	62 0x3E >	63 0x3F ?
4x	64 0x40 @	65 0x41 A	66 0x42 B	67 0x43 C	68 0x44 D	69 0x45 E	70 0x46 F	71 0x47 G	72 0x48 H	73 0x49 I	74 0x4A J	75 0x4B K	76 0x4C L	77 0x4D M	78 0x4E N	79 0x4F O
5x	80 0x50 P	81 0x51 Q	82 0x52 R	83 0x53 S	84 0x54 T	85 0x55 U	86 0x56 V	87 0x57 W	88 0x58 X	89 0x59 Y	90 0x5A Z	91 0x5B [	92 0x5C \	93 0x5D)	94 0x5E ^	95 0x5F _
6x	96 0x60 `	97 0x61 a	98 0x62 b	99 0x63 c	100 0x64 d	101 0x65 e	102 0x66 f	103 0x67 g	104 0x68 h	105 0x69 i	106 0x6A j	107 0x6B k	108 0x6C l	109 0x6D m	110 0x6E n	111 0x6F o
7x	112 0x70 p	113 0x71 q	114 0x72 r	115 0x73 s	116 0x74 t	117 0x75 u	118 0x76 v	119 0x77 w	120 0x78 x	121 0x79 y	122 0x7A z	123 0x7B {	124 0x7C 	125 0x7D }	126 0x7E ~	127 0x7F DEL

Abbildung 3: ASCII-Tabelle

a) 48 61 6C 6C 6F 20 57 65 6C 74 00

"abc": ...

n	$n+1$	$n+2$	$n+3$
a	b	c	0

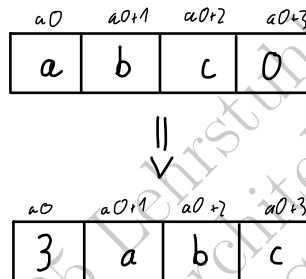
 ...

Das Ende eines Strings wird standardmäßig mit einem NULL-Byte gekennzeichnet (sog. C-Strings).

Hinweis: Ein `char` (1 Byte groß!) ist ein C-Datentyp, der oft zum Speichern von ASCII-Zeichen verwendet wird (aber nicht muss). Ein String ist dann ein `char`-Array. Eine vollständige Übersicht über die in ERA verwendeten Datentypen finden Sie in der dritten Zentralübung.

- Machen Sie sich theoretisch klar, wie man den String „Hallo Welt“ per Hand in ASCII umwandeln würde. Sie finden eine ASCII-Tabelle in Abbildung 3.
- Ein alternativer Weg, Strings darzustellen, ist ein Längenpräfix. Diese werden auch Pascal-Strings genannt. Dabei wird die Länge des Strings explizit im ersten Byte des Strings abgespeichert.

Schreiben Sie ein Unterprogramm, welches NULL-terminierte Strings zu Pascal-Strings umwandelt. Die Startadresse des Strings werde in `a0` übergeben. Sie dürfen davon ausgehen, dass ausreichend Speicher reserviert wurde um den Pascal-String an der gleichen Speicherstelle abzuspeichern.



Bonusaufgabe: Schreiben Sie ein Unterprogramm, welches Pascal-Strings in NULL-terminierte Strings umwandelt. Die Argumente sollen gleich wie in der vorherigen Teilaufgabe übergeben werden.

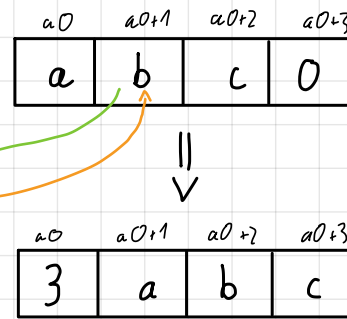
- Überlegen Sie sich mögliche Vor- und Nachteile von Pascal-Strings im Vergleich zu NULL-terminierten Strings.

4 optional: Taschenrechner-Tester (Präsenzaufgabe 01)

Bearbeitung der Präsenzaufgabe 01 (nicht bewertet) auf <https://artemis.tum.de/courses/516>

to-pascal:

; a0: start address
 ; t0: current address
 ; t1: current char
 ; t2: previous char



mv t0, a0

loop:

lbu t1, 0(t0) 1

sb t2, 0(t0) 2

beq zero, t1, finished oder: beqz t0, finished

mv t2, t1 3

addi t0, t0, 1

j loop

finished:

sub t0, t0, a0

sb t0, 0(a0)

ret

c) Vorteil: Die Länge ist direkt ablesbar. Somit können bestimmte Funktionen effizienter programmiert werden.

Nachteil: Länge auf 255 begrenzt. Wenn die Länge sich ändert, muss Präfix aktiv angepasst werden.

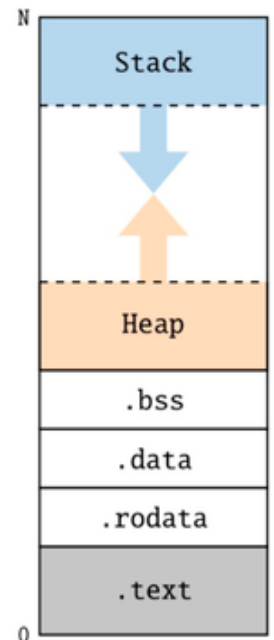
4 optional: Taschenrechner-Tester (Präsenzaufgabe 01)

Bearbeitung der Präsenzaufgabe 01 (nicht bewertet) auf <https://artemis.tum.de/courses/516>.

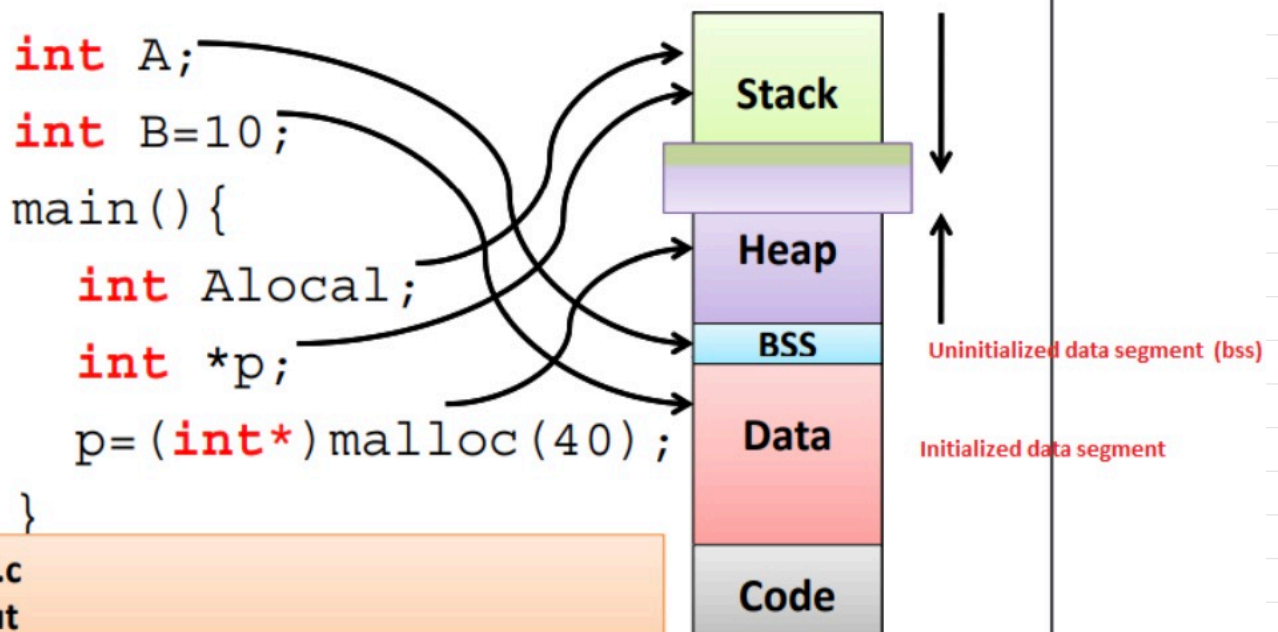
Speicherlayout

wichtige sog. Sections:

1. `.text`
beinhaltet ausführbaren Code
2. `.data`
beinhaltet globale, schreibbare, initialisierte Daten
3. `.rodata`
beinhaltet globale, nicht-schreibbare, initialisierte Daten
4. `.bss` (= block starting symbol)
beinhaltet globale, schreibbare, nicht-initialisierte Daten
effektiv nur ein Marker für „hier kommen n Bytes“



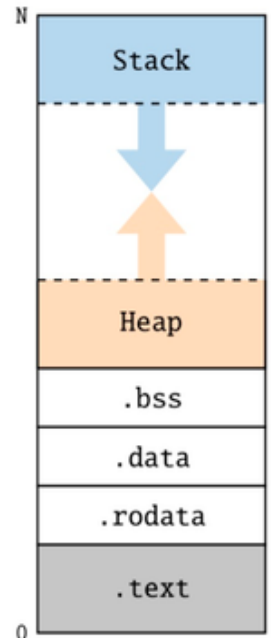
Memory layout of C program



Speicherlayout

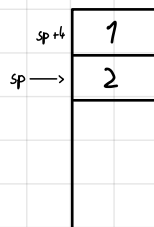
wichtige sog. Sections:

1. `.text`
beinhaltet ausführbaren Code
2. `.data`
beinhaltet globale, schreibbare, initialisierte Daten
3. `.rodata`
beinhaltet globale, nicht-schreibbare, initialisierte Daten
4. `.bss` (= block starting symbol)
beinhaltet globale, schreibbare, nicht-initialisierte Daten
effektiv nur ein Marker für „hier kommen n Bytes“



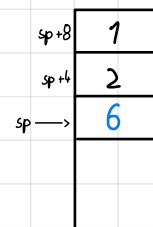
Stack:

Push $\times 5$ ($\times 5 = 6$):



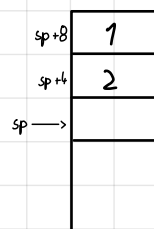
LIFO:

Pop $\times 5$:



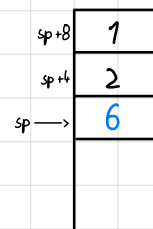
1. SP erniedrigen:

`addi sp, sp, -4`



1. Daten Laden:

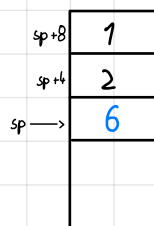
`lw  $\times 5$ , 0(sp)`



$\times 5 = 6$

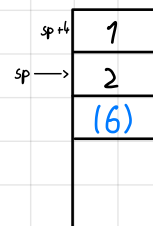
2. Daten speichern:

`sw  $\times 5$ , 0(sp)`



2. SP erhöhen:

`addi sp, sp, 4`



5 Palindromtest (Hausaufgabe 03)

Bearbeitung und Abgabe der Hausaufgabe 03 auf <https://artemis.tum.de/courses/516> bis Sonntag, den 09.11.2025, 23:59 Uhr.

Unterprogrammaufrufe in RISC-V (Basic)

0. Register sichern, Parameter vorbereiten `mv s0, ra`
1. Sprung (Jump and Link) `jal ra, function / call function`
2. Operation ...
3. Rücksprung (Jump (and Link) Register) `jalr zero, 0(ra) / ret`
4. Register wiederherstellen `mv ra, s0`

© 2025 Lehrstuhl für
Rechnerarchitektur &
Parallele Systeme
alle Rechte vorbehalten

Referenzmaterial

Name	Register Number	Use
zero	x0	Constant value 0
ra	x1	Return address
sp	x2	Stack pointer
gp	x3	Global pointer
tp	x4	Thread pointer
t0-2	x5-7	Temporary registers
s0/fp	x8	Saved register/Frame pointer
s1	x9	Saved register
a0-1	x10-11	Function arguments/Return values
a2-7	x12-17	Function arguments
s2-11	x18-27	Saved registers
t3-6	x28-31	Temporary registers

Abbildung 1: RISC-V 32-Bit Register

op	funct3	funct7	Type	Instruction	Description	Operation
0000011 (3)	000	–	I	lb rd, imm(rs1)	load byte	rd = SignExt([Address] _{7:0})
0000011 (3)	001	–	I	lh rd, imm(rs1)	load half	rd = SignExt([Address] _{15:0})
0000011 (3)	010	–	I	lw rd, imm(rs1)	load word	rd = [Address] _{31:0}
0000011 (3)	100	–	I	lbu rd, imm(rs1)	load byte unsigned	rd = ZeroExt([Address] _{7:0})
0000011 (3)	101	–	I	lhu rd, imm(rs1)	load half unsigned	rd = ZeroExt([Address] _{15:0})
0010011 (19)	000	–	I	addi rd, rs1, imm	add immediate	rd = rs1 + SignExt(imm)
0010011 (19)	001	0000000*	I	slli rd, rs1, uimm	shift left logical immediate	rd = rs1 << uimm
0010011 (19)	010	–	I	slti rd, rs1, imm	set less than immediate	rd = (rs1 < SignExt(imm))
0010011 (19)	011	–	I	sltiu rd, rs1, imm	set less than imm. unsigned	rd = (rs1 < SignExt(imm))
0010011 (19)	100	–	I	xori rd, rs1, imm	xor immediate	rd = rs1 ^ SignExt(imm)
0010011 (19)	101	0000000*	I	srlr rd, rs1, uimm	shift right logical immediate	rd = rs1 >> uimm
0010011 (19)	101	0100000*	I	srair rd, rs1, uimm	shift right arithmetic imm.	rd = rs1 >>> uimm
0010011 (19)	110	–	I	ori rd, rs1, imm	or immediate	rd = rs1 SignExt(imm)
0010011 (19)	111	–	I	andir rd, rs1, imm	and immediate	rd = rs1 & SignExt(imm)
0010111 (23)	–	–	U	auipc rd, upimm	add upper immediate to PC	rd = {upimm, 12'b0} + PC
0100011 (35)	000	–	S	sb rs2, imm(rs1)	store byte	[Address] _{7:0} = rs2 _{7:0}
0100011 (35)	001	–	S	sh rs2, imm(rs1)	store half	[Address] _{15:0} = rs2 _{15:0}
0100011 (35)	010	–	S	sw rs2, imm(rs1)	store word	[Address] _{31:0} = rs2
0110011 (51)	000	0000000	R	add rd, rs1, rs2	add	rd = rs1 + rs2
0110011 (51)	000	0100000	R	sub rd, rs1, rs2	sub	rd = rs1 – rs2
0110011 (51)	001	0000000	R	sll rd, rs1, rs2	shift left logical	rd = rs1 << rs2 _{4:0}
0110011 (51)	010	0000000	R	slt rd, rs1, rs2	set less than	rd = (rs1 < rs2)
0110011 (51)	011	0000000	R	sltu rd, rs1, rs2	set less than unsigned	rd = (rs1 < rs2)
0110011 (51)	100	0000000	R	xor rd, rs1, rs2	xor	rd = rs1 ^ rs2
0110011 (51)	101	0000000	R	srl rd, rs1, rs2	shift right logical	rd = rs1 >> rs2 _{4:0}
0110011 (51)	101	0100000	R	sra rd, rs1, rs2	shift right arithmetic	rd = rs1 >>> rs2 _{4:0}
0110011 (51)	110	0000000	R	or rd, rs1, rs2	or	rd = rs1 rs2
0110011 (51)	111	0000000	R	and rd, rs1, rs2	and	rd = rs1 & rs2
0110111 (55)	–	–	U	lui rd, upimm	load upper immediate	rd = {upimm, 12'b0}
1100011 (99)	000	–	B	beq rs1, rs2, label	branch if =	if (rs1 == rs2) PC = BTA
1100011 (99)	001	–	B	bne rs1, rs2, label	branch if ≠	if (rs1 ≠ rs2) PC = BTA
1100011 (99)	100	–	B	blt rs1, rs2, label	branch if <	if (rs1 < rs2) PC = BTA
1100011 (99)	101	–	B	bge rs1, rs2, label	branch if ≥	if (rs1 ≥ rs2) PC = BTA
1100011 (99)	110	–	B	bltu rs1, rs2, label	branch if < unsigned	if (rs1 < rs2) PC = BTA
1100011 (99)	111	–	B	bgeu rs1, rs2, label	branch if ≥ unsigned	if (rs1 ≥ rs2) PC = BTA
1100111 (103)	000	–	I	jalr rd, rs1, imm	jump and link register	PC = rs1 + SignExt(imm), rd = PC + 4
1101111 (111)	–	–	J	jal rd, label	jump and link	PC = JTA, rd = PC + 4

*Encoded in instr_{31:25}, the upper seven bits of the immediate field

Abbildung 2: RISC-V 32-Bit Integerbefehle